

编辑器工具

本页面介绍在 Unity 编辑器中辅助 MIDI 开发的一组工具。

工具	模式	菜单
MIDI 监视器	仅 Play 模式	Window > MIDI > Monitor
虚拟 MIDI 控制器	仅 Play 模式	Window > MIDI > Virtual Controller
SMF 预览	仅 Edit 模式	Window > MIDI > SMF Preview
Project Settings	Edit	Edit > Project Settings > MIDI

MIDI 监视器

在 Play 模式下实时以列表形式显示 MIDI 输入/输出消息的编辑器窗口。它提供相当于 DAW MIDI 监视器的功能，可提升开发时的调试效率。

注意：此工具**仅在 Play 模式下可用**。在 Edit 模式（非 Play）下不会显示 MIDI 消息。

打开方式

Window > MIDI > Monitor

显示内容

列	内容
Time	相对于 Play 开始的相对时刻 (mm:ss.fff)
Dir	IN (输入) / OUT (输出)
Device	设备 ID (缩略显示, 通过工具提示显示完整内容)
Ch	MIDI 通道
Type	消息类型
Detail	音符编号、CC 编号、音名等

初始版本中显示的消息

输入 (IN)

- Note On / Note Off
- Control Change (CC)
- Program Change (PC)
- 设备连接 / 断开事件

输出 (OUT)

- Note On / Note Off
- Control Change (CC)
- Program Change (PC)

对于 Note 类消息, Detail 列还会通过 MidiNoteUtility 显示音名 (例如 C4 (60))。

工具栏

操作	说明
Clear	清空日志
Export	将应用过滤后的日志导出为 CSV 文件
Auto Scroll	在添加新消息时自动滚动
Max Lines	保留的最大行数（默认 1000，最大 10000）
Ch 1-16	在 0-15 / 1-16 之间切换通道显示

过滤器

过滤器	选项
Direction	All / IN / OUT
Device	已连接设备列表（All = 全部设备）
Ch	All / 0-15（或 1-16 显示）
Type	All / NoteOn / NoteOff / CC / PC / Device
Search	对 Detail 列、Device 列进行部分匹配搜索

过滤器设置会保存到 `EditorPrefs` 中，下次打开窗口时仍会保留。

工作原理

输入消息的捕获

在 Play 模式开始时会自动生成 `[MIDI Monitor Proxy]` GameObject，并作为事件处理器注册到 `MidiManager`。接收到的 MIDI 事件会被追加到监视器的日志缓冲区中。

输出消息的捕获

在 `MidiManager` 的发送方法（Note On/Off、CC、PC）执行时，会经由编辑器专用钩子（`#if UNITY_EDITOR`）记录 OUT 日志。来自 `MidiSend`（Fluent API）的发送同样会被显示。

Play 模式结束时

Proxy GameObject 会被自动销毁，并从 `MidiManager` 解除注册。日志缓冲区的内容在 Play 结束后仍会保留在窗口中（可在下次 Play 时清空）。

推荐用法

1. 打开 `Window > MIDI > Monitor`
2. Play 示例场景（ `Assets/MIDI/Samples/Scenes/` ）
3. 连接 MIDI 设备，或从脚本通过 `MidiSend.To(...).NoteOn(...)` 发送
4. 查看 IN / OUT 日志

发送测试示例：

```
using jp.kshoji.unity.midi.util;  
  
// 在 Play 期间执行  
MidiSend.ToFirstOutput().Channel(0).NoteOn(60, 127);
```

虚拟 MIDI 控制器

无需物理 MIDI 设备，即可从编辑器发送 Note / CC / Program Change / Pitch Bend 的模拟器。

打开方式

Window > MIDI > Virtual Controller

功能

- 2 个八度键盘（C3–B4）
- 16 个 CC 滑块（CC 编号保存在 EditorPrefs 中）
- 通道 (1–16) / 组 / 力度设置
- 输出设备选择（未连接时会自动创建 `editor:virtual-controller` 虚拟输出）
- All Notes Off / Panic (CC 120 + 123)

使用方法

1. Play 一个会调用 `MidiManager.InitializeMidi()` 的场景（例如示例场景）
2. 打开 `Window > MIDI > Virtual Controller`
3. 操作键盘或 CC 滑块
4. 在 `Window > MIDI > Monitor` 中查看 OUT 消息

SMF 预览 / 导入

在 Edit 模式下加载 `.mid` 文件，查看轨道与事件列表，并可导出为 `MidiSequenceAsset`。

打开方式

```
Window > MIDI > SMF Preview  
Assets > Import MIDI File...
```

功能

- 拖放加载 `.mid` 文件
- 轨道列表 / 事件列表（Tick、时刻、类型、Detail）
- 速度、拍号、轨道名等元信息显示
- 基于文本的音符列表（Note Roll）
- `MidiSequenceAsset` / JSON 导出

导出的 `MidiSequenceAsset` 可以使用 [SMF 工具](#) 的 `SmfPlayer` 播放。

Project Settings

从 `Edit > Project Settings > MIDI` 管理 MIDI Plugin 的全局设置。

设置项

分区	项目
Devices	<code>defaultInputDeviceId</code> / <code>defaultOutputDeviceId</code> （空 = 自动选择）
Bluetooth MIDI	<code>autoScanBleOnInit</code> 、 <code>bleScanTimeoutMs</code> （0 = 无限制）
RTP-MIDI	<code>rtpMidiPort</code> （默认 5004）、 <code>rtpMidiSessionName</code>
Debug	<code>logLevel</code> （None / Error / Warning / Info / Verbose）、 <code>enableMidiMonitorOnPlay</code>
Development	<code>developmentBuildOnlyVerboseLog</code>

首次导入时会自动生成 `Assets/MIDI/Resources/MidiProjectSettings.asset` 。在 `MidiManager.InitializeMidi()` 完成时，BLE 自动扫描以及 RTP-MIDI 服务器启动设置会生效。

未来的编辑器功能（尚未实现）

功能	概要
设备浏览器	Window > MIDI > Device Browser — 设备列表、Vendor/Product ID、测试发送、复制 ID
Scene View 调试叠加层	在 Play 期间于场景上显示 MIDI 状态
延迟测量（RTT 工具）	编辑器发送→接收往返 UI（用于 BLE / RTP-MIDI 评估）。运行时敲击校准已可通过 Foundation 的 MidiLatencyCalibrator 使用

源文件

文件	作用
MidiMonitorWindow.cs 等	MIDI 监视器
VirtualMidiControllerWindow.cs	虚拟控制器
SmfPreviewWindow.cs / SmfImportUtility.cs	SMF 预览
MidiProjectSettings.cs	设置 ScriptableObject
MidiProjectSettingsProvider.cs	Project Settings UI
MidiManager.ProjectSettings.cs	运行时联动

相关文档

- [SMF 工具 \(SmfPlayer / MidiRecorder\)](#)
- [工具类 \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [关于编辑器与生命周期的注意事项](#)
- [入门指南](#)
- [示例项目](#)